

TP2 - R2.08

Toutes les réponses avec explications doivent être sur le même notebook jupyter à déposer à la fin de la séance sur moodle.

Listes

Définir la liste : `liste = [17, 38, 10, 25, 72]`, puis effectuez les actions suivantes :

- triez et affichez la liste;

```
liste = [17, 38, 10, 25, 72]
liste.sort()
print("Liste triée :", liste)
```

```
# # - triez et affichez la liste;

liste = [17, 38, 10, 25, 72]
liste.sort()
print("Liste triée :", liste)
```

✓ 0.0s

Liste triée : [10, 17, 25, 38, 72]

- ajoutez l'élément 12 à la liste et affichez la liste;

```
liste.append(12)
print("Liste après ajout de 12 :", liste)
```

```
# ajoutez l'élément 12 à la liste et affichez la liste;
liste.append(12)
print("liste après ajout de 12 :", liste)
```

✓ 0.0s

liste après ajout de 12 : [10, 17, 25, 38, 72, 12]

- renversez et affichez la liste;

```
liste.reverse()
print("Liste renversée :", liste)
```

```
# renversez et affichez la liste;
liste.reverse()
print("liste renversée:", liste)
```

✓ 0.0s

liste renversée: [12, 72, 38, 25, 17, 10]

- affichez l'indice de l'élément 17;

```
index_17 = liste.index(17)
print("Indice de 17 :", index_17)
```

```
index_17 = liste.index(17)
print("Indice de 17:", index_17)
✓ 0.0s
Indice de 17: 4
```

- enlevez l'élément 38 et affichez la liste;
liste.remove(38)
print("Liste après suppression de 38 :", liste)

```
liste.remove(38)
print("liste après suppression de 38:", liste)
✓ 0.0s
liste après suppression de 38: [12, 72, 25, 17, 10]
```

- affichez la sous-liste du 2e au 3e élément;

```
print("Sous-liste du 2e au 3e élément :", liste[1:3])
print("Sous-liste du 2e au 3e élément:", liste[1:3])
Sous-liste du 2e au 3e élément: [72, 25]
```

- affichez la sous-liste du début au 2e élément;

```
print("Sous-liste du début au 2e élément :", liste[:2])
print("Sous-liste du début au 2e élément :", liste[:2])
Sous-liste du début au 2e élément : [12, 72]
```

- affichez la sous-liste du 3e élément à la fin de la liste;

```
print("Sous-liste du 3e élément à la fin :", liste[2:])
print("Sous-liste du 3e élément à la fin:", liste[2:])
Sous-liste du 3e élément à la fin: [25, 17, 10]
```

- affichez la sous-liste complète de la liste;

```
print("Liste complète :", liste[:])
```

```
print("liste complète :", liste[:])
```

```
liste complète : [12, 72, 25, 17, 10]
```

- affichez le dernier élément en utilisant un indigage négatif.

```
print("Dernier élément :", liste[-1])
```

```
print("Dernier élément :", liste[-1])
```

```
Dernier élément : 10
```

- Supposons qu'on a deux listes, une qui contient des prénoms et l'autres des noms.
- Écrivez une fonction qui parcours simultanément les deux listes et affiche les prénoms et noms côte à côte.

```
prenoms = ["Jeanne", "Anne", "Camille"]
```

```
noms = ["Papin", "Bakayoko", "Drogba"]
```

Supposons qu'on a deux listes, une qui contient des prénoms et l'autres des noms.

Ecrivez une fonction qui parcours simultanément les deux listes et affiche les prénoms et noms côte à côte.

```
prenoms = ["Jeanne", "Anne", "Camille"]
noms = ["Papin", "Bakayoko", "Drogba"]
```

```
# Supposons qu'on a deux listes, une qui contient des prénoms et l'autres des noms.
# Ecrivez une fonction qui parcours simultanément les deux listes et affiche les prénoms et noms côte à côte.
def afficher_noms_complets(prenoms, noms):
    for prenom, nom in zip(prenoms, noms):
        print(prenom, nom)

prenoms = ["Jeanne", "Anne", "Camille"]
noms = ["Papin", "Bakayoko", "Drogba"]

afficher_noms_complets(prenoms, noms)
```

✓ 0.0s

```
Jeanne Papin
Anne Bakayoko
Camille Drogba
```

2. Compter les occurrences des mots dans une chaîne

On utilise un dictionnaire pour stocker le nombre d'apparitions de chaque

```
# 2. Compter les occurrences des mots dans une chaîne

def compte_occurrences(chaine):
    mots = chaine.split()
    occurrences = {}
    for mot in mots:
        occurrences[mot] = occurrences.get(mot, 0) + 1
    return occurrences

print(compte_occurrences("la vie est belle c'est la vie"))
```

✓ 0.0s

```
{'la': 2, 'vie': 2, 'est': 1, 'belle': 1, "c'est": 1}
```

3. Somme des valeurs d'un dictionnaire

Cette fonction prend un dictionnaire en entrée et retourne la somme de ses valeurs.

```
# Somme des valeurs d'un dictionnaire

def somme_valeurs(dictionnaire):
    return sum(dictionnaire.values())

dico = {'a': 10, 'b': 20, 'c': 5}
print(somme_valeurs(dico))
```

✓ 0.0s

```
35
```

4. Afficher les fichiers .conf et leur nombre de lignes

On liste les fichiers **.conf** d'un répertoire et affiche leur nombre de lignes.

```
# 4. Afficher les fichiers .conf et leur nombre de lignes

import os

def afficher_fichiers_conf(repertoire):
    try:
        for fichier in os.listdir(repertoire):
            chemin_fichier = os.path.join(repertoire, fichier)
            if fichier.endswith(".conf") and os.path.isfile(chemin_fichier):
                try:
                    with open(chemin_fichier, 'r', encoding='utf-8') as f:
                        nb_lignes = sum(1 for _ in f)
                        print(f"{fichier}: {nb_lignes} lignes")
                except Exception as e:
                    print(f"Impossible de lire {fichier}: {e}")
            except FileNotFoundError:
                print("Répertoire introuvable.")
            except PermissionError:
                print("Permission refusée, essayez avec sudo.")

# Exemple d'appel
afficher_fichiers_conf("/etc")
```

✓ 0.0s

```
logrotate.conf: 27 lignes
fuse.conf: 17 lignes
swtpm-localca.conf: 4 lignes
xattr.conf: 22 lignes
idmapd.conf: 10 lignes
ntopng.conf: 16 lignes
ld.so.conf: 2 lignes
swtpm_setup.conf: 4 lignes
deluser.conf: 20 lignes
kernel-img.conf: 4 lignes
mke2fs.conf: 43 lignes
digilent-adept.conf: 40 lignes
nftables.conf: 15 lignes
nfs.conf: 86 lignes
request-key.conf: 43 lignes
nsswitch.conf: 22 lignes
ldap.conf: 302 lignes
sysctl.conf: 68 lignes
hdparm.conf: 125 lignes
sudo_logsrvd.conf: 243 lignes
resolv.conf: 5 lignes
debconf.conf: 83 lignes
appstream.conf: 29 lignes
Impossible de lire nslcd.conf: [Errno 13] Permission denied: '/etc/nslcd.conf'
```

5. Nombre de fichiers dans un répertoire

On compte le nombre de fichiers présents dans un répertoire donné.

```
# 5. Nombre de fichiers dans un répertoire

import os

def compter_fichiers(repertoire):
    return len([f for f in os.listdir(repertoire) if os.path.isfile(os.path.join(repertoire, f))])

print(compter_fichiers("/etc"))
```

✓ 0.0s Python

127

6. Gestion des erreurs dans une division

Correction de la fonction **division** pour gérer l'erreur de division par zéro.

```
> def division(a, b):
    try:
        return a // b
    except ZeroDivisionError:
        return "Erreur : Division par zéro impossible."

# Tests
print(division(10, 2))
print(division(10, 0))
```

[36] ✓ 0.0s

... 5
Erreur : Division par zéro impossible.

7. Expressions régulières

a) Explication des symboles des regex

- **.** : correspond à n'importe quel caractère sauf une nouvelle ligne.
- **^** : indique le début d'une chaîne.
- **\$** : indique la fin d'une chaîne.
- ***** : zéro ou plusieurs répétitions du caractère précédent.
- **+** : une ou plusieurs répétitions du caractère précédent.
- **?** : zéro ou une répétition du caractère précédent.
- **{n}** : exactement **n** répétitions.
- **{n, }** : au moins **n** répétitions.
- **{n, m}** : entre **n** et **m** répétitions.
- **[]** : définit un ensemble de caractères.
- **** : caractère d'échappement.
- **|** : opérateur "ou".

- **()** : groupe de capture.

b) Test d'une regex en Python

```
import re

match = re.match(r"GR(?:S)", "GRIS")
print(match is not None) # Devrait afficher True si la correspondance est trouvée
```

✓ 0.0s

c) Vérification d'un numéro de téléphone

On utilise une expression régulière pour vérifier si un numéro de téléphone est valide (format français : 10 chiffres).

```
import re

def verifier_numero_telephone(numero):
    pattern = r"^0[1-9]([-. ]?\d{2}){4}$"
    return bool(re.match(pattern, numero))

# Tests
print(verifier_numero_telephone("06 12 34 56 78")) # True
print(verifier_numero_telephone("0612345678"))     # True
print(verifier_numero_telephone("+33612345678"))    # False
print(verifier_numero_telephone("123456"))          # False
```

✓ 0.0s

True
True
False
False

Explication du regex **`^0[1-9]([-. ]?\d{2}){4}$`**:

- ☒ **`^0[1-9]`** : commence par un 0 suivi d'un chiffre entre 1 et 9.
- ☒ **`([-. ]?\d{2}){4}`** : 4 groupes de 2 chiffres, éventuellement séparés par -, .
\$: fin de la chaîne.

